Extended Abstract

Motivation LLM pre-training has limitations on how it can learn from data. We aim to explore whether we can reuse pre-training data at test-time through retrieval and additional test-time compute to further improve models on a wide variety of downstream tasks.

Method We augment the reader model with retrieval from pre-training datasets, consistency, and query rewriting. Consistency comes in the form of both self-consistency, and our newly introduced inter-document consistency (consistency as a reranker across documents). We also explore query rewriting by fine-tuning the reader model with GRPO on question-document pairs, where the rewards are the embedding similarity between the rewritten query and the golden document.

Implementation We use Llama 3.1 8B Instruct and Qwen 3 8B as reader models, FAISS for indexing, GTE Qwen 2 1.5B as the embedding model. We initially implemented GRPO from scratch, but found it difficult to integrate with FSDP, so ended up using TRL. We use a modified version of OpenAI's SimpleEvals with vLLM to run and evaluate experiments. We used FineWeb-edu as the main dataset in our datastore, and augmented it with AutoMathText, FineMath, OpenWebMath, and StackMathQA. All of these are commonly used pre-training datasets. We used question-document pairs from BRIGHT and a synthetic dataset for GRPO.

Results With inter-document consistency, we are able to achieve 78.9 on MMLU, 77.6 on Math 500, and 75.3 on AIME25. This is a lift of 7.6, 18.3, 8.5 percentage points over the reader model baselines, respectively. Inter-document consistency contributed a significant improvement over the retrieval-only and self-consistency-only baselines. On the other hand, we found that using GRPO improved query rewriting over using the reader model as the rewriter, but query rewriting overall did not lead to better performance. Analysis suggests that the GRPO model has moved too far away from the base model, and performs poorly when the fine-tuning and test data are not similar enough.

Discussion While we have shown improvements on a couple of tasks, we can expand this further to a broader set such as SimpleQA, GPQA, and code generation. We would also like to improve on our existing work by trying additional rewards, comparing GRPO to rejection sampling with fine-tuning, and exploring alternative ways to aggregate results.

Conclusion Using retrieval with additional test-time compute through inter-document consistency can significantly improve performance on both knowledge and reasoning tasks. While there is definitely room to improve on each of the individual methods we have explored, we also hope researchers consider other approaches for reusing pre-training datasets.

Enhancing Retrieval of Pre-Training Data with More Test-Time Compute

Alex Fang
Department of Computer Science
Stanford University
alexfang@stanford.edu

Abstract

Large language models learn from their vast pre-training corpora to gain the ability to solve an ever increasing variety of tasks; yet, pre-training data is still underutilized. In this work, we show that retrieving from pre-training datasets at test time can lead to large gains on both knowledge and reasoning tasks. We explore combining retrieval with simple forms of test-time compute, including self-consistency, inter-document consistency, and query rewriting, to achieve gains of 7.6 percentage points on MMLU with a Llama 3.1 8B reader model and 8.5 percentage points on AIME25 with a Qwen3 8B reader model. Although initial query rewriting results are negative, analysis suggests that there is room for improvement through iterating on the fine-tuning dataset and crafting better rewards.

1 Introduction

Large language models (LLMs) have consistently improved performance by scaling pre-training compute. In parallel to scaling, researchers have also made significant efforts to improve both the architectures and the datasets used by these LLMs.

While LLMs are able to solve an incredible number of tasks, in their current form they have several limitations. For example, they are unable to generalize in the example of the reversal curse (Berglund et al., 2023), and struggle with long-tail knowledge (Kandpal et al., 2023). Additionally, the scaling trend is often described as log-linear, meaning that larger amounts of compute are necessary to make the same gains at larger scales.

Taking advantage of the effort put into creating these sophisticated pre-training datasets, we explore whether reusing them through retrieval at test-time can further improve performance. Additionally, retrieval can be supplemented with additional test-time compute to further improve the generations. Retrieval and test-time compute are a natural fit, as we can explore different forms of consistency to parallelize test-time compute, as well as query rewriting to serialize test-time compute.

We evaluate our methods on a variety of downstream tasks that cover multiple domains and measure both knowledge and reasoning. Our experiments demonstrate that through retrieval and inter-document consistency, a Llama 3.1 8B model can achieve 78.9% on MMLU and 77.6% on Math 500, while a Qwen 3 8B model can achieve 75.3% on AIME25.

Our initial query rewriting results are less promising. While GRPO can improve a query rewriting model, all of our query rewriting experiments perform worse than using the original question as the query. However, our analysis suggests that we may be able to improve results by changing our fine-tuning dataset, or crafting better rewards.

In the near future, we hope to expand these results by evaluating on additional tasks such as SimpleQA and code generation, improving query rewriting with better designed rewards, and exploring alternative ways to aggregate results.

2 Related Work

Prior work such as Lewis et al. (2020) and Ram et al. (2023) have shown that retrieval augmented generation can improve performance on knowledge intensive benchmarks. More recently Shao et al. (2024a) demonstrated that scaling up the datastore can reliably further improve performance.

Recent work (Snell et al., 2024; Brown et al., 2024) has shown that scaling test-time compute can be an efficient way of improving LLM performance. Specifically, they suggest parallelizing inference compute across trials, and sequentially iterating on a model's output. Results can then be aggregated with techniques like self-consistency (Wang et al., 2022) or verifiers. Retrieval is naturally suited to both as it can parallelize across the documents retrieved, while sequentially iterating over the search query to improve the retrieved documents.

Hsu et al. (2024) aims to improve the latter by using DSPy prompt optimizers to generate multi-hop datasets explored using reinforcement learning. While we also explore using reinforcement learning for query rewriting, we differ from this work by focusing on pre-training evaluation tasks instead of multi-hop retrieval, retrieving from pre-training data rather than a specialized dataset, and using a different reinforcement learning setup.

Asai et al. (2023) take a different approach by fine-tuning on interleaved passages annotated with a critic model to use reflection tokens to iterate on retrieval. However, recent work such as Guo et al. (2025) show that reasoning can naturally occur without specialized tokens. We allow the off-the-shelf reader model to determine whether the document in-context is useful, and by retrieving from a much larger corpus we hope that the retrieval system is much more likely to find relevant documents.

Lastly, there has been an increase in Deep Research web-browsing agents from companies like Google, OpenAI, and Perplexity. We choose to focus on retrieval because we believe that retrieval may be a better augmentation of LLMs than proprietary search engines. While web-browsing agents use search engines as black-box tools, retrieval may be customized towards LLMs rather than traditional search engine optimization goals.

3 Method

We start with simple neural-embedding based similarity for retrieval and pre-pend the documents to the evaluation questions during generation. This requires tuning the number of retrieved documents because more documents contain more information, but also can have distracting information the overcrowds the context. We choose not to use a lexical retriever to perform hybrid retrieval due to scaling considerations that come with using pre-training sized datasets. We then use additional test-time compute to further improve performance.

The simplest form of test-time compute is self-consistency (Wang et al., 2022). Every question is answered n times in parallel, and we choose the most common answer as the final answer. Wang et al. (2022) demonstrate this this improves chain-of-thought reasoning on reasoning and arithmetic benchmarks. While our evaluations mostly have limited answer choices (e.g. multiple choice, integers) that allow for regex based comparisons, this is not a crucial limitation of this method because the reader model can also function as a judge, as demonstrated in Chen et al. (2023).

We combine the ideas of consistency with retrieval by introducing inter-document consistency. As shown in Figure 1, we apply self-consistency for each retrieved document separately, and then use the consistency count of each document as a reranker to determine the quality of the documents. We then take the self-consistency generated answer from the best document as our final answer. The idea is that better documents should make generation more consistent. While this significantly increases inference costs, this can be easily parallelized, and provides additional gains over using various LLM-based rerankers.

Lastly, we explore query rewriting with both the base model and with a fine-tuned version using GRPO (Shao et al., 2024b). GRPO builds off the ideas of PPO (Schulman et al., 2017), but substitutes the

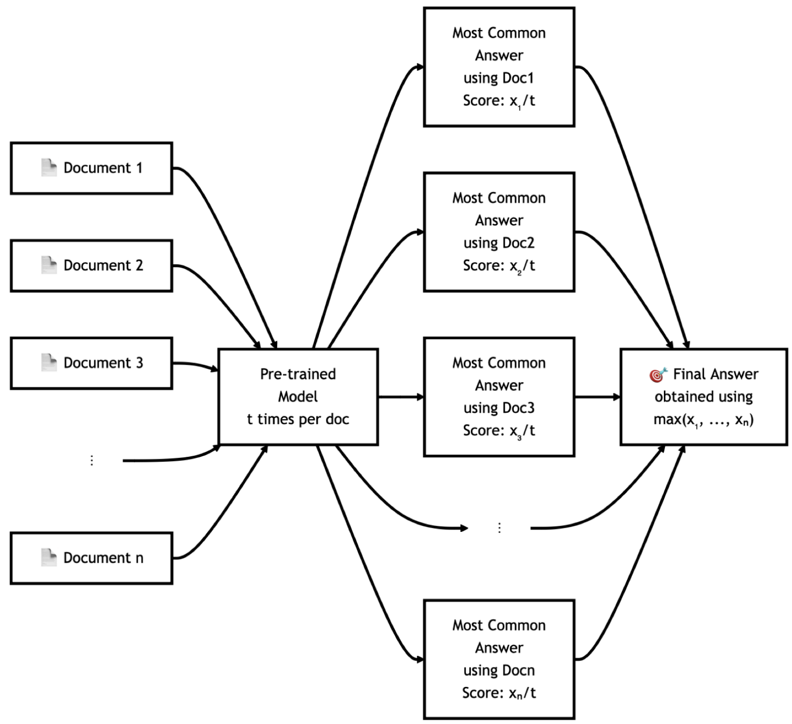

Figure 1: Our most successful form of test-time compute comes from inter-document consistency. We apply self-consistency on generating while retrieving from individual documents, and select the answer from the most self-consistent document.

critic model with taking multiple samples from a prompted LLM and scoring them with handcrafted rewards. We use question-document pairs and ask the model to generate a rewritten query, then calculate the rewards using the embedding similarity between the rewrite and the document. We choose to use GRPO due to our access to a handcrafted reward that should be correlated with our intermediate goal of retrieving good documents, its past success towards improving LLMs, and its simplicity.

4 Experimental Setup

We use Llama 3.1 8B Instruct (Grattafiori et al., 2024) as the reader model for MMLU and MATH-500 (Lightman et al., 2023), Qwen 3 8B (Yang et al., 2025) as the reader model for AIME, and GTE Qwen2 1.5B Instruct (Li et al., 2023) as the embedding model. Our retrieval system uses FAISS (Johnson et al., 2019) for indexing.

The main dataset we are retrieving from is FineWeb-edu (Penedo et al., 2024) because it is one of the better open-source pre-training datasets for LLMs while being smaller (for compute and iteration purposes) than the alternatives. We then supplemented this with frequently used math pre-training datasets, specifically choosing AutoMathText (Zhang et al., 2025), FineMath (Allal et al., 2025), OpenWebMath (Paster et al., 2023), StackMathQA (Zhang, 2024).

We choose to initially focus evaluation on MMLU (Hendrycks et al., 2020) and Math 500 (Lightman et al., 2023) because they are pre-training benchmarks that the reader model has room for improvement on, may require reasoning, and can benefit from both retrieval and test-time compute. We then added AIME25 as a harder reasoning task, which required a stronger reader model. One additional benefit of AIME25 is that it was released later than our retrieval datasets, thus reducing the chance of contamination.

For GRPO, we use TRL and accelerate to ease integration with FSDP. We fine-tune the model with GRPO on question-document pairs, coming from two difference sources: 1. the BRIGHT benchmark

(Su et al., 2024) consisting of pairs that are challenging for standard retrieval techniques, and 2. synthetic generations using Claude Sonnet 4.

5 Results

5.1 Main Retrieval and Consistency Results

Table 1: Comparing baseline reader model performance against retrieval with a variety of test-time compute options. MMLU and Math 500 results use Llama 3.1 8B instruct as the reader model, while AIME25 results use Qwen 3 8B as the reader model.

Method	MMLU STEM	MMLU Humanities	MMLU Social Sci	MMLU Other	MMLU All	Math 500	AIME25
Baseline	0.705	0.640	0.774	0.769	0.713	0.593	0.668
Retrieval	0.730	0.678	0.833	0.805	0.752	0.656	0.662
Self Consistency	0.759	0.678	0.812	0.798	0.753	0.688	0.700
Retrieval Consistency	**0.788**	**0.720**	**0.853**	**0.827**	**0.789**	**0.776**	**0.753**

In Table 1, we see that retrieval and self-consistency both improve over the baseline model, and that inter-document (retrieval) consistency improves upon all of the above. This applies not only to knowledge tasks such as MMLU, but also reasoning tasks like Math 500 and AIME25. We ensure that the inter-document consistency results are compute matched with the self-consistency results, and note that it has different behavior than applying self-consistency directly to retrieval.

These results rely on having a strong enough base model. A model that cannot perform a task at all will not be able to perform it from retrieval alone. For example, we use Qwen 3 instead of Llama 3.1 on AIME25 because Llama 3.1 has close to 0% accuracy with and without retrieval.

While there is a possibility for contamination, we believe these concerns should be minor. In experiments with AIME24, we see that exact contamination would lead to very high retrieval results, and it would be unlikely that inter-document consistency significantly improves upon it.

5.2 Rewriting Results

Our query rewriting results are less promising. In Table 2, while using GRPO improves upon using the base model for rewriting, it is worse than the baseline that doesn't use query rewriting. Qualitatively, Llama rewrites seem to acknowledge that it is trying to write a query, but GRPO rewrites are mixed and sometimes even attempt to directly answer the question it should be trying to rewrite. While attempting to answer the question may actually be a good query, this suggests that GRPO is moving too far away from the base model and may be losing its instruction following ability. We tried tuning the KL divergence parameter for GRPO, but the rewritten queries are still mixed, while the average reward is much lower.

5.3 GRPO Rewards Analysis

We do additional analysis on our GRPO results by looking across the average reward across datasets. In Table 3, we see that GRPO can improve average rewards for in distribution data, but fails to generalize to out of distribution data. Specifically, GRPO on synthetic data sees higher average rewards on itself as well as on a held out validation set while seeing minimal improvement on BRIGHT, and GRPO on BRIGHT sees higher average rewards on itself while seeing minimal

Table 2: While GRPO may slightly improve query rewriting over rewriting with the base model, current query rewriting results do not improve over the question as query baseline.

Method\MMLU	STEM	Humanities	Social Science	Other	All
Baseline	**0.730**	0.678	**0.833**	**0.805**	**0.752**
Llama Rewrites	0.714	0.668	0.809	0.802	0.740
GRPO on Bright Data Rewrites	0.714	0.661	0.811	0.799	0.737
GRPO on Synthetic Data Rewrites	0.718	**0.665**	0.818	0.804	0.742

improvement on the synthetic datasets. This suggests that we may need to expand our fine-tuning dataset, or generate synthetic data that is more in distribution to our evaluation sets.

Connecting this back to MMLU, the distribution shift appears to be in the content difference rather than the format difference. Using just the question rather than the entire multiple choice with options does not improve performance on MMLU.

Table 3: Using query-document embedding similarity as the reward for GRPO improves average reward for in distribution data, but not for out of distribution data.

Method	Bright Pairs	Synthetic Train	Synthetic Validation
Question	0.488	0.568	0.569
Llama Rewrite	0.450	0.509	0.509
GRPO on Bright Data Rewrites	**0.621**	0.579	0.580
GRPO on Synthetic Data Rewrites	0.482	**0.649**	**0.626**

6 Discussion and Future Work

We hope to expand these results by evaluating on additional tasks such as SimpleQA, GPQA, and code generation, improving query rewriting with better designed rewards, and exploring alternative ways to aggregate results.

Expanding our evaluation set is key to demonstrating that our methods can apply to all LLM tasks. In particular, SimpleQA focuses on long-tail knowledge, GPQA focuses on scientific reasoning, while code generation is a completely different type of task. We would also like to better understand the different modes in which retrieval benefits knowledge tasks compared to reasoning tasks. We can compare SimpleQA with Math evaluations and ablate the retrieved data to analyze whether reasoning tasks are reasoning over the retrieved documents instead of simply using facts.

While query-document embedding similarity is a key condition for retrieving the best document, it ignores the interaction with other documents. We can try to craft a better reward by using contrastive methods to increase the score of the positive document while decreasing the scores of negative documents. We can also utilize existing answers or generate synthetic answers and reward the model for being able to utilize the document to obtain the correct final answer.

Beyond rewards, we can also try to improve query rewriting by increasing the size of our synthetic dataset, using rejection sampling with fine-tuning which may be more stable with small datasets, or using a stronger base model.

Lastly, we hope to build on our results by investigating whether logits can be used as a substitute for consistency. Consistency is a very simple form of test-time compute and is also compute intensive. Initial ideas to further improve upon it include using signals from logits to determine aggregation weighting or determine confidence.

7 Conclusion

We have shown that retrieval on pre-training datasets with additional test-time compute can improve LLM capabilities. Although retrieval cannot fully replace a pre-trained model, it can augment it and shore up some of its weaknesses. Pre-training and retrieval both benefit from the same datasets, and we hope that future work can find additional methods that reuse pre-training data to further improve LLMs.

8 Team Contributions

- **Alex Fang:** Solo project, did all of the work, received advising from advisor, project mentor, etc.

Changes from Proposal No major changes, but the RL component (GRPO on query rewriting) was less successful than we had hoped. We also changed the evaluation from SimpleQA / MMLU / GPQA to MMLU / Math 500 / AIME25, but eventually plan on expanding our evaluation set.

Code Most code was hand written. For GRPO, I wrote initial code for the RL algorithm before giving it to a LLM to help with distributed training (which required additional manual debugging). I also used a LLM to help with the TRL/FSDP version.

References

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. 2025. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. arXiv:2502.02737 [cs.CL] `https://arxiv.org/abs/2502.02737`

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*.

Lukas Berglund, Meg Tong, Max Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomasz Korbak, and Owain Evans. 2023. The Reversal Curse: LLMs trained on" A is B" fail to learn" B is A". *arXiv preprint arXiv:2309.12288* (2023).

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787* (2024).

Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. 2023. Universal self-consistency for large language model generation. *arXiv preprint arXiv:2311.17311* (2023).

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).

Sheryl Hsu, Omar Khattab, Chelsea Finn, and Archit Sharma. 2024. Grounding by trying: Llms with reinforcement learning-enhanced retrieval. *arXiv preprint arXiv:2410.23214* (2024).

Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. 2023. Large language models struggle to learn long-tail knowledge. In *International Conference on Machine Learning*. PMLR, 15696–15707.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. 2023. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281* (2023).

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*.

Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. 2023. OpenWebMath: An Open Dataset of High-Quality Mathematical Web Text. arXiv:2310.06786 [cs.AI]

Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin A Raffel, Leandro Von Werra, Thomas Wolf, et al. 2024. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems* 37 (2024), 30811–30849.

Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics* 11 (2023), 1316–1331.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

Rulin Shao, Jacqueline He, Akari Asai, Weijia Shi, Tim Dettmers, Sewon Min, Luke Zettlemoyer, and Pang Wei W Koh. 2024a. Scaling retrieval-based language models with a trillion-token datastore. *Advances in Neural Information Processing Systems* 37 (2024), 91260–91299.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. 2024b. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314* (2024).

Hongjin Su, Howard Yen, Mengzhou Xia, Weijia Shi, Niklas Muennighoff, Han-yu Wang, Haisu Liu, Quan Shi, Zachary S Siegel, Michael Tang, et al. 2024. Bright: A realistic and challenging benchmark for reasoning-intensive retrieval. *arXiv preprint arXiv:2407.12883* (2024).

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

Yifan Zhang. 2024. StackMathQA: A Curated Collection of 2 Million Mathematical Questions and Answers Sourced from Stack Exchange. `https://huggingface.co/datasets/math-ai/StackMathQA`

Yifan Zhang, Yifan Luo, Yang Yuan, and Andrew Chi-Chih Yao. 2025. Autonomous Data Selection with Zero-shot Generative Classifiers for Mathematical Texts. *The 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025 Findings)* (2025).